



10

jQuery

Interactions & Framework

“Write less, do more”

Compatibilité & simplification

Créée en 2006, **jQuery est une bibliothèque JavaScript** développée afin de simplifier l'écriture du code, la manipulation du DOM et la compatibilité entre navigateurs.

- Très populaire grâce à sa syntaxe simple et efficace
- A facilité l'utilisation des animations, effets visuels et requêtes AJAX
- Perte de vitesse face aux nouvelles librairies telles que React, Vue.js, Angular, etc.
- Son utilisation reste importante pour maintenir de nombreux sites web et plugins
- Reste utile pour comprendre les bases du JavaScript

Le javascript a depuis bien évolué, certaines pratiques devenant plus facile en pure JS qu'en jQuery, comme quoi, la balance peut toujours s'inverser !

Intégration

Script & CDN

Pour intégrer **jQuery**, deux choix s'offrent à nous :

- Via CDN
 - En important directement leur librairie dans notre projet
- (Je vous laisse deviner la meilleure pratique... ;D)*

Import via CDN :

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

Vérification :

```
$(document).ready(function() {  
    console.log("jQuery fonctionne !");  
});
```

Rem. : `$(function() {...});` et `$(document).ready(function(){...});` veulent dire exactement la même chose.

Sélecteurs et manipulation du DOM

Syntaxe des sélecteurs CSS :

`$("p")` // tous les `<p>`

`$("#id")` // élément avec id

`$(".classe")` // éléments avec la classe

Manipulations :

`$("#titre").text("Nouveau titre");`

`$(".bloc").css("color", "red");`

`$("img").attr("alt", "nouvelle image");`

Quelques fonctions jQuery

- `$(".section a").addClass("ma-classe")` : ajoute la classe « ma-classe »
- `$(".section a").removeClass("ma-classe")` : enlève la classe
- `$(".section a").attr("href")` : renvoie la valeur de l'attribut (href)
- `$(".section a").data("index")` : renvoie la valeur de l'attribut data-index
- `$(".section a").on("click", function() { /* ... */ })` :
écoute [les évènements](#) sur le *click*

Bonus :

event.preventDefault() : fonction javascript permettant de désactiver le comportement par défaut de l'élément pointé.

Évènements & animations

Gestion des événements

// Évènement classique

```
$("#bouton").click(function() {  
    alert("Bouton cliqué !");  
});
```

// Pratique plus moderne

```
$("#bouton").on("click", function() {  
    $(this).toggleClass("actif");  
});
```

Animations

// Effet de fondu

```
$("#texte").fadeOut();
```

// Effet d'apparition/glisement (sous-menu)

```
$("#menu").slideToggle();
```

// Effet de mouvement

```
$(".carte").animate({ opacity: 0.5, left: "50px" },  
500);
```

Quand je ...,

- On cherche à déterminer un **évènement** et **quand** il se produit

```
$("#selecteur").on("click", function() {  
    // do something  
});
```

.on permet la gestion de plusieurs évènements mais aussi de l'étendre à tout élément qui serait ajouté dynamiquement et donc encore inexistant.

C'est une fonction propre à jQuery nécessitant deux paramètres : L'évènement et les actions associées.

Le composant ...

- On s'attend à ce que le composant **réagisse** et affiche le résultat

```
$("#selecteur").on("click", function() {  
    $(this).hide(); // hide this  
});
```

function(params) {} permet d'exécuter un ensemble de tâches sur des paramètres.

this fait quant à lui directement référence au sélecteur qui le précède.



Mise en pratique !

- Aller à l'énoncé,
 - Si vous utilisez codepen.io forkez (en bas à droite)
 - Sinon copier-coller dans les fichiers adéquats (html, css, js)
 - <https://getbootstrap.com/docs/5.3/getting-started/introduction/#starter-template>
n'oubliez pas d'ajouter vos fichiers js et css
- Déchiffrez le code, relever les indices que j'ai laissé dans les différents langages

Aller plus loin

- [Documentation jQuery](#) (en)
- [You Might Not Need jQuery](#) (JS Power - en)
- [jQuery Cheat Sheet](#) (en)

Pratique

- [W3Schools jQuery Ex.](#) (en)
- [Codeacademy](#) (en)